

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..6

ADMIN OPERATIONS

 Copy as Markdown

Email admin operations

The email identity provider provides admin operations through `EmailIdpAdmin` for managing email accounts and cleaning up expired or dangling requests. These operations are useful for administrative tasks, maintenance, and preventing database bloat.

Accessing the `EmailIdpAdmin`

You can access the admin operations through the `AuthServices.instance.emailIdp` property:

```
import 'package:serverpod_auth_idp_server/providers/email.dart';
import 'package:serverpod_auth_idp_server/core.dart';

// Get the EmailIdp instance
final emailIdp = AuthServices.instance.emailIdp;

// Access admin operations
final admin = emailIdp.admin;
```

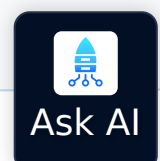
Account management

The admin API provides methods for managing email accounts:

Finding accounts

```
// Find an account by email
final account = await admin.findAccount(
  session,
  email: 'user@example.com',
);
```

Creating accounts



```
// Create an email authentication account
final emailAccountId = await admin.createEmailAuthentication(
  session,
  authUserId: userId,
  email: 'user@example.com',
  password: 'securePassword123',
);
```

Deleting accounts

```
// Delete an account by email
await admin.deleteEmailAccount(
  session,
  email: 'user@example.com',
);

// Delete all email accounts for a user
await admin.deleteEmailAccountByAuthUserId(
  session,
  authUserId: userId,
);
```

Setting passwords

```
// Set or update a password for an account
await admin.setPassword(
  session,
  email: 'user@example.com',
  password: 'newSecurePassword123',
);
```

WARNING

The `setPassword` method does not validate the password against the configured password policy. Make sure to validate the password before calling this method if needed.

Finding active account requests

You can also check for active account requests:

```
final accountRequest = await admin.findActiveEmailAccountRequest(
  session,
  accountRequestId: requestId,
);
```

This is useful for checking the status of a registration request or verifying if a request is still valid.

Cleanup operations

Over time, expired account requests, password reset requests, and failed login attempts can accumulate in the database, leading to database bloat and potential performance issues. It's important to periodically clean these up to prevent database bloat. Such requests are not automatically cleaned up since they can be useful for auditing purposes, so it is up to each application to decide when to clean them up.

Cleaning up expired account requests

Account requests that have expired (users who started registration but never completed it) should be cleaned up:

```
// Delete all expired account requests
await admin.deleteExpiredAccountRequests(session);

// Delete a specific account request
await admin.deleteEmailAccountRequestById(
  session,
  requestId,
);
```

Cleaning up expired password reset requests

Password reset requests that have expired (users who requested a password reset but never completed it) should be cleaned up:

```
// Delete all expired password reset requests
await admin.deleteExpiredPasswordResetRequests(session);

// Delete password reset request attempts for a specific email
// Useful when you want to allow a user to request a new password
// even though they have hit the rate limit
await admin.deletePasswordResetRequestsAttemptsForEmail(
  session,
  email: 'user@example.com',
);
```

Cleaning up failed login attempts

Failed login attempts, tracked for rate limiting, should also be cleaned up when no longer useful for auditing purposes:

```
// Delete all failed login attempts
await admin.deleteFailedLoginAttempts(session);

// Delete failed login attempts older than a specific duration
await admin.deleteFailedLoginAttempts(
  session,
  olderThan: Duration(days: 30),
);
```

